

Rigorously Certified Neural-Network Computation of Nonlinear Water Waves at Near-Machine Precision

Michal Shavit^{1*}, Yongji Wang¹ and Tristan Buckmaster¹

^{1*}Courant Institute of Mathematical Sciences, New York University,
NY, 10012, USA.

*Corresponding author(s). E-mail(s): ms14479@nyu.edu;

Abstract

We develop a computational framework that combines machine-learning-based discovery with rigorous computer-assisted numerical certification and formal verification for strongly nonlinear partial differential equations. We demonstrate the approach for periodic gravity–capillary traveling water waves. Physics-informed neural networks are used as a search mechanism, with the wave profile, wave speed and, in continuation experiments, surface tension learned directly from the governing equations without training data. A single formulation recovers three classical high-amplitude regimes: pure-gravity Stokes waves, pure-capillary Crapper waves and gravity–capillary Wilton ripples. Starting from a high-amplitude Wilton ripple, continuation in surface tension further reveals new high-amplitude candidate solutions. We freeze one such machine-learned candidate as an explicit finite representation and certify its residual using interval arithmetic, rigorously accounting for floating-point roundoff, Fourier truncation, aliasing and grid-to-continuum errors. The resulting computation provides a rigorous uniform measure of how accurately the candidate satisfies the governing equation, reaching near-machine precision. We then formalize the mathematical structure of the certification argument in Lean, which machine-checks the deductions connecting the certified numerical bounds to the final residual theorem. Both the interval certification and formalization were developed with substantial assistance from a large language model, while the validity of the result remains independent of the language model: the numerical bounds are rigorously enclosed by interval arithmetic and the subsequent mathematical deductions are checked by the Lean kernel. Together, these steps provide a pipeline from neural discovery to rigorous numerical certification and machine-checked mathematical verification, enabling

machine learning to explore strongly nonlinear regimes while retaining explicit and auditable mathematical guarantees.

Keywords: keyword1, Keyword2, Keyword3, Keyword4

Lean verification and certified computations

Contents

1	Introduction	2
2	Results	4
2.1	Neural discovery and recovery of high-amplitude water waves	6
2.1.1	Continuation experiments and new solutions	7
2.2	Second stage training and machine precision candidate	11
2.3	Certified residual estimates and formal verification in Lean	12
2.3.1	Rigorous certification by interval arithmetic	13
2.3.2	Formal verification in Lean	14
3	Discussion	15
4	Methods	16
4.1	PINN architecture	16
4.1.1	Neural representation	16
4.1.2	Residual, Hilbert transform and loss	17
4.1.3	Collocation and optimization	17
4.2	Second-stage training	18
4.2.1	Gradient-normalized residual weighting	19
4.3	Computer-assisted accuracy proof	20
4.3.1	Computer-assisted interval certification of the numerical residual	21
4.3.2	Formal definitions, analytic estimates and numerical certificate in Lean	23
4.3.3	Certified profile, computational settings and bounds	24
4.4	Reproducibility and file map	25
5	Data and code availability	26

1 Introduction

Machine learning can produce numerical solutions of nonlinear partial differential equations with remarkably small residuals, but small residuals alone do not provide a mathematical guarantee of accuracy. Here we develop a computational framework that bridges this gap by combining machine-learning-based discovery with rigorous numerical certification and formal verification. We demonstrate the approach

for strongly nonlinear, high-amplitude water waves, where physics-informed neural networks discover candidate solutions with residuals approaching the limits of double-precision arithmetic. We then convert a selected candidate into a rigorous uniform residual certificate using interval arithmetic and formally verify the mathematical deductions underlying the certificate in Lean. The resulting pipeline provides a route from machine-learned numerical discovery to certified, machine-checkable scientific computation.

Surface gravity–capillary waves constitute a classical and enduring problem of fluid mechanics. Their governing equations are remarkably rich, describing phenomena ranging from small-amplitude capillary ripples to extreme ocean waves, yet are notoriously difficult to analyze because of their full nonlinearity and nonlocality. The subject has a long history spanning oceanography, mathematical physics and rigorous analysis, from Stokes’ expansions and Zakharov’s Hamiltonian formulation to modern results on well-posedness and asymptotics [1]; see [2] for a review. Despite this progress, the strongly nonlinear, high-amplitude regime remains substantially more difficult to access. In this regime, perturbative descriptions cease to apply, solution branches can develop intricate structure, and even the systematic construction and accurate numerical representation of nonlinear wave solutions becomes challenging.

This combination of strong nonlinearity, nonlocality and a rich solution structure makes water waves a stringent setting in which to test whether machine learning can move beyond numerical approximation and become a tool for mathematical discovery with rigorously certified performance. We address this question through three stages: neural discovery, computer-assisted certification by interval arithmetic, and formal verification.

First, we employ physics-informed neural networks (PINNs) [3] as a search mechanism for high-amplitude two-dimensional periodic traveling solutions of the fully nonlinear water-wave equations. The network is optimized directly to satisfy the equations in conformal variables, with the wave profile, wave speed, and, in selected continuation experiments, the surface-tension parameter learned simultaneously. In this way, the network parametrizes and explores nonlinear solution branches. Building on the two-stage training procedure of ref. [4, 5], we obtain neural representations whose residuals approach the limits of double-precision arithmetic. Using the same formulation, we recover three canonical high-amplitude regimes: pure-gravity Stokes waves [6], pure-capillary Crapper waves [7] and mixed gravity–capillary Wilton ripples [8]. Starting from a high-amplitude Wilton ripple and continuing in surface tension, the network further discovers new high-amplitude candidate solutions of the governing equation.

Second, we turn numerical accuracy into a rigorous statement using interval arithmetic. A small residual obtained by floating-point computation does not itself constitute a rigorous certificate, because roundoff, truncation, aliasing and discretization errors enter through the nonlinear and nonlocal operations in the water-wave equations. We therefore freeze a selected machine-learned candidate as an explicit finite representation and propagate rigorous intervals through the governing operators, explicitly accounting for floating-point roundoff, Fourier truncation, aliasing errors arising from grid-based evaluation, and the error incurred in passing from a finite grid

to the continuous domain. This yields a certified uniform upper bound on the residual at near-machine precision, rather than a small residual observed only at finitely many numerical evaluation points.

Third, we formally verify the mathematical structure of the certification argument in Lean, a proof assistant and functional programming language. At this level of accuracy, the numerical certificate is itself necessarily computer assisted: our interval-arithmetic proof is implemented in Julia using the Arb library and consists of hundreds of lines of code that propagate rigorous bounds through nonlinear and nonlocal operations. Although the resulting computation is mathematically rigorous, its implementation is too extensive and delicate to be audited line by line through conventional peer review. We therefore use Lean to provide a machine-checkable logical layer for the certification argument. Lean verifies the mathematical identities and deductions connecting the certified numerical bounds to the final residual theorem, substantially reducing the amount of computational reasoning that must be manually audited. The numerical interval bounds constitute the numerical trust boundary, while the subsequent mathematical deductions are checked by the Lean kernel.

Both the interval-arithmetic certification and its formalization were developed with substantial assistance from a large language model. Crucially, however, the validity of the resulting certificate does not depend on the reliability of the language model: the numerical inequalities are rigorously enclosed by interval arithmetic, while the mathematical deductions built upon them are checked by the Lean kernel. This separation allows a large language model to assist substantially in constructing a complex computer-assisted proof without becoming part of the trusted mathematical foundation of the result.

Together, these developments establish a pipeline from machine-learning-based discovery to rigorous numerical certification and machine-checked mathematical verification. Pairs of these ingredients have previously been combined—including neural discovery with computer-assisted proof and rigorous numerics with formal verification—but, to our knowledge, the complete chain has not previously been closed from a solution discovered by a neural network, through rigorous interval certification, to formal verification of the resulting certificate in a proof assistant. The framework therefore provides a route for using machine learning not only as a scientific-computing tool, but also as an instrument of mathematical discovery: exploring strongly nonlinear regimes that are difficult to access analytically while retaining explicit, rigorous and auditable mathematical guarantees.

2 Results

Surface gravity-capillary waves are described in terms of the Euler equation for an incompressible, irrotational fluid of infinite depth with potential velocity $\mathbf{v} = \nabla\phi$ and a free surface $y = \eta(x, t)$, where x is the horizontal coordinate and y is the vertical coordinate. Incompressibility gives the Laplace equation $\Delta\phi = 0$ inside the fluid domain $\mathcal{D} = \{(x, y), -\infty < y \leq \eta(x, t)\}$ with the boundary conditions at the

surface $y = \eta(x, t)$:

$$(\partial_t + \mathbf{v} \cdot \nabla)(y - \eta(x, t)) = 0, \quad (1)$$

$$\phi_t + \frac{1}{2}|\nabla\phi|^2 + g\eta = \kappa \frac{\eta_{xx}}{(1 + \eta_x^2)^{3/2}}, \quad (2)$$

and $\nabla\Phi \rightarrow 0$ as $y \rightarrow -\infty$. Here g is the acceleration of gravity, κ is the surface tension coefficient and

$$\frac{\eta_{xx}}{(1 + \eta_x^2)^{3/2}} = \partial_x \left(\frac{\eta_x}{\sqrt{1 + \eta_x^2}} \right) \quad (3)$$

is the mean curvature of the free surface. The first boundary condition is the kinematic condition, expressing that the free surface moves with the fluid particles. The second is Bernoulli's condition, in which the pressure jump across the free surface is proportional to the mean curvature.

For periodic waves traveling in one direction, one can reformulate the equations above in terms of a conformal map $z(w, t) := x(w, t) + iy(w, t)$ that maps the lower half-plane $\{(u, v) \in [-\pi, \pi] \times (-\infty, 0]\}$ to the fluid domain $\mathcal{D}$ as illustrated in Fig. 1. This approach is described in [9–11]. On $v = 0$, steady traveling waves $y = y(u - ct)$ solve the Babenko equation

$$\begin{aligned} \mathcal{B}(y) = & -\mu \hat{H}y_u + \left(y + y \hat{H}y_u + \hat{H}(yy_u) \right) \\ & -\beta \partial_u \left(\frac{y_u}{\sqrt{y_u^2 + (1 + \hat{H}y_u)^2}} \right) + \beta \hat{H} \partial_u \left(\frac{1 + \hat{H}y_u}{\sqrt{y_u^2 + (1 + \hat{H}y_u)^2}} \right) = 0, \end{aligned} \quad (4)$$

where μ is the squared (rescaled) wave speed, $\beta = \kappa/(g\lambda^2)$ is the Bond number encoding surface tension, gravity, and wavelength λ . $y(u)$ is the elevation of the free-surface in conformal coordinates and $\hat{H}$ is the periodic Hilbert transform defined on $L_2[-\pi, \pi]$ by linearity from the relation

$$\hat{H}e^{inu} = -i \text{sign}(n)e^{inu}. \quad (5)$$

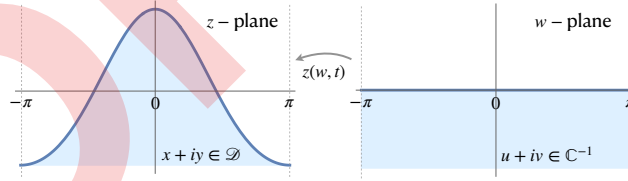


Fig. 1 The conformal mapping from the region in the w plane $(u, v) \in [-\pi, \pi] \times (-\infty, 0]$ to the domain occupied by the fluid in the (x, y) plane, with $(x, y) \in [-\pi, \pi] \times (-\infty, \eta(x, t)]$. The line $v = 0$ (blue line) is mapped onto the free surface $\eta(x, t)$.

The Babenko equation (4) admits two families of known high amplitude solutions at the extreme ends of zero gravity or zero capillarity. At $\beta = 0$, the one parameter family of waves is the classical Stokes waves represented as a curve in the speed and amplitude plane (c, a) connecting small amplitude solutions with a limiting Stokes wave of extreme height with a $2\pi/3$ crest angle [6]. The amplitude is defined as the difference between the maximal and minimal height divided by the period

$$a = \frac{\max y - \min y}{2\pi}. \quad (6)$$

At zero gravity, $g = 0$, there is a family of pure capillary solutions, found by Crapper, which is represented as a curve in the speed and amplitude plane (c, a) connecting small amplitude capillary waves with a limiting Crapper wave of extreme height that encloses a bubble.

When both gravity and capillarity are present, resonant families of solutions, known as Wilton ripples, arise at special values of the surface tension. At the linear level, a Wilton ripple contains two harmonics, $\cos(mu)$ and $\cos(nu)$, whose phase speeds coincide. Although small-amplitude Wilton ripples have been computed numerically, their high-amplitude regime is much less understood both analytically and numerically.

2.1 Neural discovery and recovery of high-amplitude water waves

We use physics-informed neural networks (PINNs) [3] to discover water-wave profiles and their associated physical parameters directly from the Babenko equation (4). Rather than being trained on precomputed or experimental data, the network is optimized against the governing equation together with the physical and normalization constraints defining the solution family under consideration. The same neural architecture and residual-based optimization framework is used throughout, while the physical parameters and auxiliary constraints are adapted to select the relevant branch. The trained profile is denoted by y_N . For each run, we minimize a weighted squared residual of the Babenko equation and, when used, its differentiated form, together with auxiliary conditions controlling quantities such as vertical translation, amplitude and symmetry. The wave speed $c_N = \sqrt{\mu_N}$ is optimized simultaneously with the profile. Depending on the experiment, gravity and surface tension are fixed to select a particular physical regime, while in the continuation experiments the surface-tension parameter is itself allowed to vary. Details of the network architecture, Hilbert-transform evaluation, loss function, constraints and optimization procedure are given in Methods.

We first use three canonical high-amplitude solution families as benchmarks for the discovery framework (Fig. 2). These regimes probe qualitatively different balances in the same nonlinear equation. In the zero-surface-tension limit, $\beta = 0$, the network reproduces the classical pure-gravity Stokes branch [12]. In the zero-gravity limit, $g = 0$, it recovers the pure-capillary Crapper family [7]. When both gravity and surface tension are present, the network recovers high-amplitude (1, 2) Wilton ripples [8], for which the first and second Fourier harmonics travel at the same speed at the

linear level. The three families have markedly different geometries, dominant physical balances and Fourier structures, and their recovery therefore provides complementary tests of the same computational framework. Importantly, the appropriate physical and normalization constraints differ between these regimes; what remains common is the neural representation and equation-based optimization.

The Wilton regime provides a particularly demanding test. At small amplitude, the $(1, 2)$ Wilton branch originates from a linear resonance between two Fourier modes, but at finite amplitude their contributions are coupled by the full nonlinear water-wave equation. The amplitudes of the resonant components therefore cannot in general be varied independently: nonlinear constraints determine how the harmonics combine along the solution branch. Continuing a high-amplitude Wilton ripple is consequently not equivalent to simply varying the amplitude parameter and reoptimizing an unconstrained profile. The continuation must simultaneously preserve the appropriate nonlinear constraints while allowing the profile and wave speed parameter to adjust, making the high-amplitude resonant regime a substantially more restrictive search problem.

After reproducing a high-amplitude $(1, 2)$ Wilton ripple, we use it as a seed for neural continuation. We allow the surface-tension parameter to vary together with the wave profile and speed, while imposing the amplitude and normalization constraints required to control motion through the solution space. Starting from the same high-amplitude seed, continuation in different constrained directions generates new nearby gravity-capillary profiles with small Babenko residuals (Fig. 4). In this setting, the PINN acts as a nonlinear continuation and discovery method: it searches for profiles satisfying the governing equation.

These benchmark and continuation experiments separate two roles of the neural computation. Recovery of the Stokes, Crapper and Wilton families tests whether the framework can represent and optimize solutions across distinct nonlinear regimes when supplied with the appropriate physical constraints. Continuation away from the high-amplitude Wilton state then uses the same machinery in a discovery setting, where the resulting profiles are not prescribed in advance. The low-residual candidates obtained in this stage provide the starting point for the rigorous certification described below.

2.1.1 Continuation experiments and new solutions

After reproducing a high-amplitude $(1, 2)$ Wilton ripple, we use it as a seed for neural continuation. We allow the surface-tension parameter to vary together with the wave profile and speed, while imposing constraints on the wave amplitude and on the ratio a_1/a_2 of the two resonant Fourier components to control motion through the solution space. Starting from the same high-amplitude seed, continuation in different constrained directions produces new gravity-capillary profiles with small Babenko residuals (Fig. ??). Although these solutions are reached by continuation from a Wilton ripple, they are not simply nearby members of the known high-amplitude Wilton family: their Fourier composition and values of β depart substantially from those of the corresponding Wilton branch, and their geometry develops qualitatively different features as the continuation proceeds. Toward one end of the computed branch, the crest develops a pronounced central depression while the trough becomes increasingly

sharp, producing profiles distinct from the high-amplitude Wilton ripples used to initialize the search. To our knowledge, gravity–capillary solutions with this combination of Fourier structure, parameter values and free-surface geometry have not previously been computed.

We were also unable to continue this family toward the small-amplitude regime. Within the range accessible to our numerical continuation, the branch instead remains intrinsically finite amplitude. This observation suggests that these profiles may not arise through a conventional bifurcation from a linear water-wave mode, although establishing whether the family is genuinely disconnected from the small-amplitude spectrum requires further mathematical and numerical analysis. In this sense, the high-amplitude Wilton ripple provides an entry point into a region of the nonlinear solution landscape that is not apparent from the linearized problem.

A detailed mathematical characterization of this solution family—including its global bifurcation structure, possible limiting states, stability and connection, or lack thereof, to small-amplitude solutions—is beyond the scope of the present work. Here these solutions primarily demonstrate the exploratory capability of the computational framework: constrained neural continuation can identify previously uncharacterized high-amplitude states, which can subsequently be passed to rigorous numerical certification. Our focus in the remainder of the work is therefore on establishing this discovery-to-certification pipeline and on rigorously validating a representative machine-learned solution.

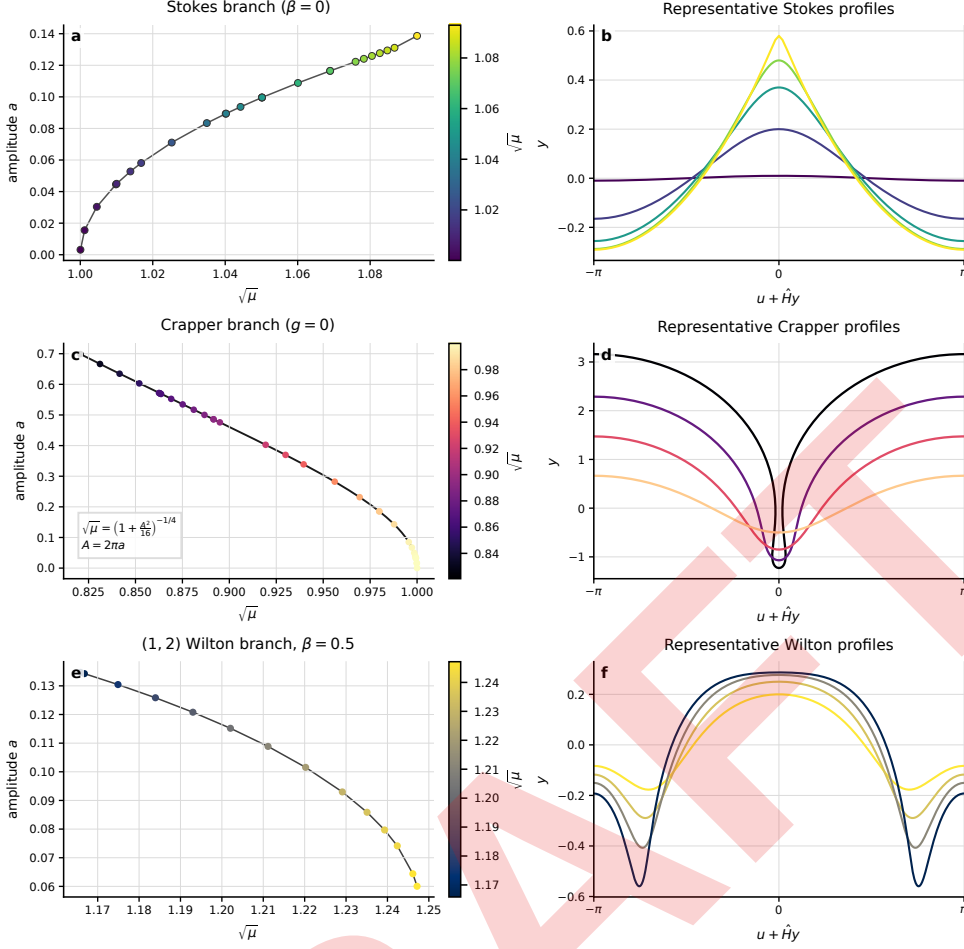


Fig. 2 Three high-amplitude regimes reproduced by the PINN. **a**, Pure-gravity Stokes branch with $\kappa = 0$, plotted in the (λ, a) plane, where $a = (\max y - \min y)/(2\pi)$. The panel shows the selected Stokes files, including the low-amplitude main-branch runs, that satisfy $\max |\mathcal{B}(y_N)| \leq 10^{-4}$, with the grey curve joining the retained runs. **b**, Five representative retained Stokes profiles plotted over $[-\pi, \pi]$ as $x + \hat{H}y$ versus y . **c**, Pure-capillary Crapper branch with $g = 0$. All Crapper files in the plotting folder are included as points on the branch, and the black curve shows the fit $\lambda = (1 + A^2/16)^{-1/4}$ with $A = 2\pi a$. **d**, Representative Crapper profiles plotted over $[-\pi, \pi]$. **e**, Reproduced (1,2) Wilton branch at $\beta = 0.5$, again with all available branch files included. **f**, Four representative Wilton free-surface profiles, including low- and high-amplitude cases, plotted over $[-\pi, \pi]$.

After reproducing the (1,2) Wilton ripple, we use it as a seed for discovery. Rather than fixing the surface tension, we train the surface-tension parameter together with the profile and speed, initializing near the high-amplitude Wilton solution. This procedure reveals nearby families of gravity-capillary waves that deform the Wilton profile

while preserving small Babenko residuals. These families form the main new solution set studied below and provide the candidates passed to the interval-arithmetic certification step.

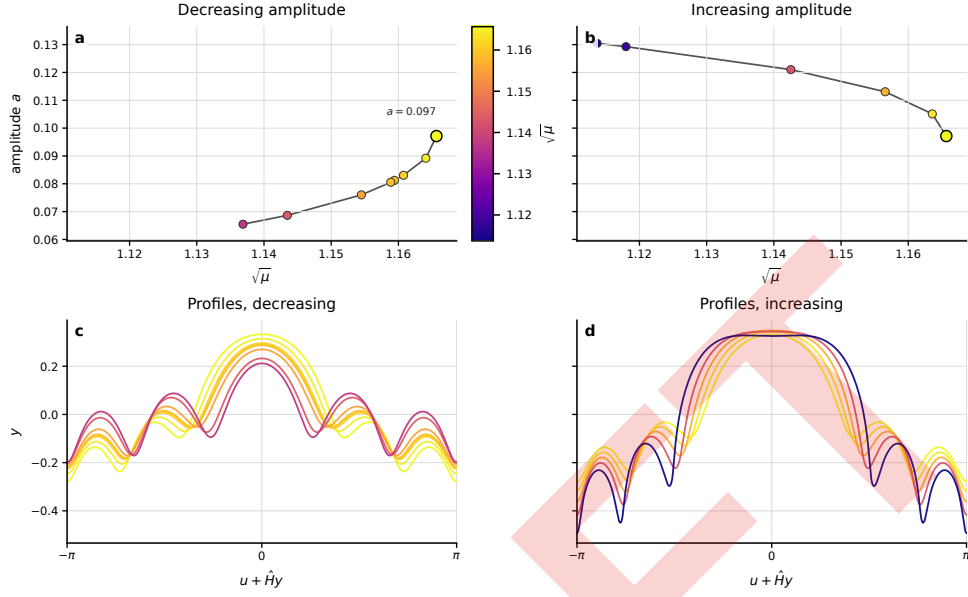


Fig. 3 Continuation away from the high-amplitude (1, 2) Wilton ripple. **a**, Amplitude-decreasing continuation at $\kappa = 0.48$, starting from the shared seed with $0.09 < a < 0.1$ and plotted in the (c, a) plane. **b**, Amplitude-increasing continuation from the same seed point. **c**, Clean profiles along the amplitude-decreasing branch. **d**, Clean profiles along the amplitude-increasing branch, with colours in both profile panels matched to the same c scale.

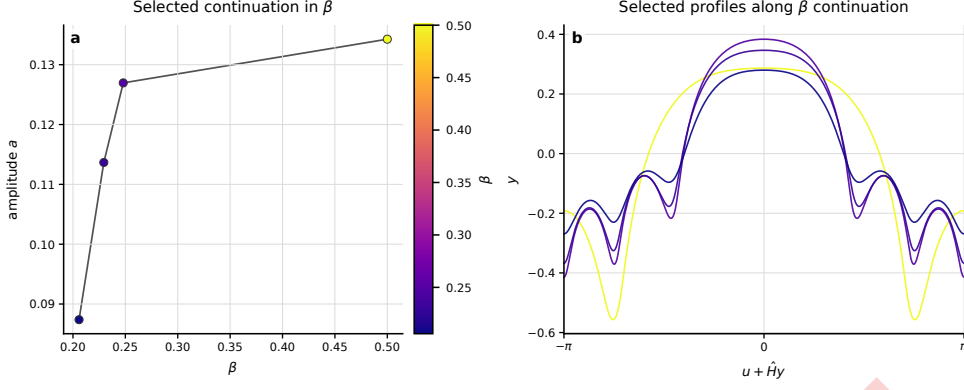


Fig. 4 Continuation near a high-amplitude Wilton ripple. **a**, Selected points on the continuation branch, shown in the amplitude- κ plane. For the three non-reference points, the stored continuation parameter is plotted as $\kappa = \beta^2$; for example, the point generated with $\beta = 0.498$ is shown at $\kappa = 0.498^2$. **b**, Corresponding free-surface profiles over one period, plotted against the conformal horizontal coordinate $x + \hat{H}y$ and colored by the same value of κ as in **a**. **c**, Certified profile at $\kappa = 0.498^2$. **d**, Pointwise residual $|\mathcal{B}(y_N)|$ for the certified profile. The dashed line marks the reported certificate level $\|\mathcal{B}(y_N)\|_\infty = 8 \times 10^{-13}$.

2.2 Second stage training and machine precision candidate

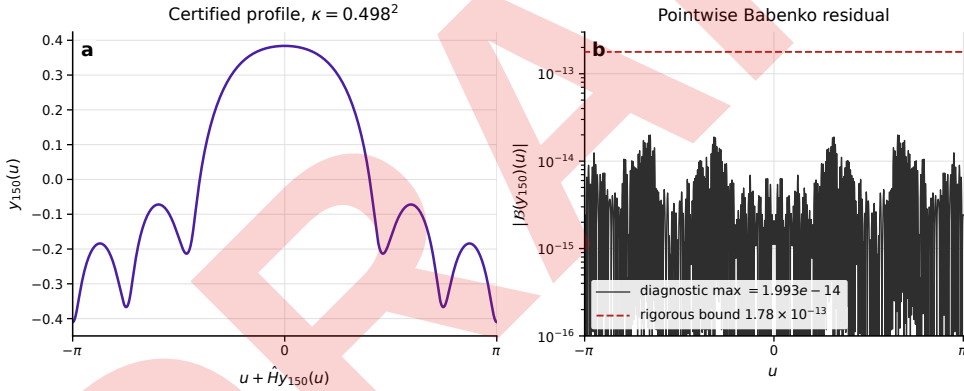


Fig. 5 Certified high-amplitude Wilton-ripple profile. **a**, Free-surface profile for the certified solution at $\beta = 0.498^2$, plotted over one period against the conformal horizontal coordinate $u + \hat{H}y$. **b**, Pointwise residual $|\mathcal{B}(y_N)|$ for the same solution. The dashed line marks the certificate level $\|\mathcal{B}(y_N)\|_\infty \leq 1.78 \times 10^{-13}$ with $N = 150$.

We demonstrate that the Babenko residual can be reduced to near the double-precision round-off level. Successive neural corrections, followed by Fourier projection and Newton polishing, reduced the maximum sampled scalar residual by approximately seven orders of magnitude, from $\mathcal{O}(10^{-7})$ to $\mathcal{O}(10^{-14})$, see Fig. 6. Independent interval validation then established a rigorous uniform residual bound of $\mathcal{O}(10^{-13})$ for the final polynomial.

2.3 Certified residual estimates and formal verification in Lean

We now turn from numerical discovery to rigorous certification. The neural-network approximation is used to construct a candidate profile; the object ultimately certified is an explicit finite trigonometric polynomial, with fixed coefficients and parameters.

Let

$$y_N(u) = \sum_{k=0}^N a_k \cos(ku), \quad (7)$$

and let $\mu, \beta > 0$ be fixed rational parameters. We write

$$R_N(u) = \mathcal{B}_{\mu, \beta}(y_N)(u) \quad (8)$$

for the residual of the gravity–capillary Babenko equation. Our objective is to establish a rigorous uniform estimate

$$\|R_N\|_{L^\infty(\mathbb{R})} = \sup_{u \in \mathbb{R}} |R_N(u)| \leq \varepsilon, \quad (9)$$

for an explicit constant ε . For the candidate certified here,

$$N = 150, \quad \varepsilon = \frac{178}{10^{15}} = 1.78 \times 10^{-13}. \quad (10)$$

This is an accuracy theorem for a supplied finite series. It is not an existence or uniqueness theorem for an exact nearby water wave. Such a result would additionally require a rigorous inverse bound for the linearized Babenko operator and a Newton–Kantorovich or contraction argument; neither ingredient is part of the present certificate.

The certification proceeds in two distinct stages. The first is a computer-assisted numerical proof based on interval arithmetic. The second is a formal verification in Lean 4 of the logic and algorithmic correctness of the interval-arithmetic proof. The trust boundary is therefore explicit: the numerical enclosures are computed by Julia through Arblib, which uses the well-audited FLINT/Arb library for rigorous real arithmetic, while Lean verifies that the certified algorithm, together with the recorded interval endpoints, implies the stated residual and denominator bounds. This separation of rigorous numerical enclosure from machine-checked logical verification is standard in computer-assisted proofs that combine interval arithmetic with proof assistants. We now describe the two stages in turn.

This certification has both practical numerical and pure-mathematical motivations. From a numerical perspective, the objective is to establish the residual accuracy of

a PINN-derived candidate through a rigorous bound, rather than through training loss, agreement with reference computations, or evaluation at finitely many points. Previous work has demonstrated numerical PINN residuals approaching the double-precision round-off floor [5], while other studies have developed a posteriori error estimates, continuous-domain residual certificates, interval-arithmetic validation, and Lean-based verification [13–16]. Numerically observing a small residual and rigorously certifying a uniform bound at the same scale are, however, distinct achievements.

Here we establish an absolute uniform residual bound of 1.78×10^{-13} for an explicit PINN-derived profile of the gravity–capillary Babenko equation, in the normalization used throughout this work. To our knowledge, this is the first rigorous continuous-domain residual certificate at this accuracy for a PINN-derived approximate solution of a nonlinear and nonlocal equation. The certification includes the numerical evaluation of the nonlinear nonlocal capillary terms, with rigorous control of Fourier truncation, aliasing, denominator positivity, and between-grid errors. It is complemented by Lean verification of the mathematical deductions from explicitly stated numerical assumptions.

From a pure-mathematical perspective, the objective is to make PINN-based discovery usable as a high-precision tool for studying highly nonlinear and nonlocal equations. The resulting computations can serve as quantitative inputs to further mathematical analysis: the candidate is explicitly defined, its residual is rigorously controlled, and the computational and logical trust boundaries are made explicit. In particular, the residual certificate supplies a first rigorous ingredient toward a computer-assisted proof of the existence of a nearby exact solution.

We now describe the two stages in turn.

2.3.1 Rigorous certification by interval arithmetic

The first stage establishes the residual bound (9) using interval ball arithmetic. All certification computations are performed in Julia using Arb, with rigorous enclosures and outward rounding, so that computed quantities are enclosed by intervals containing their exact values. In particular, the computation accounts for floating-point roundoff, Fourier truncation and aliasing errors in the evaluation of nonlinear terms, and the passage from grid-based estimates to bounds valid between grid points.

The Hilbert transform is treated differently depending on its argument. For the trigonometric polynomial y_N and its derivatives, the Hilbert transform acts exactly mode by mode and therefore introduces no spectral truncation error; numerical evaluation of the resulting finite expressions is enclosed by interval arithmetic. By contrast, nonlinear composite quantities appearing in the capillary term are not finite trigonometric polynomials. Their Hilbert transforms are evaluated through a discrete Fourier representation, with the contribution of unresolved modes and the associated aliasing errors bounded explicitly using estimates on the Fourier tail.

The capillary term also contains the denominator

$$D(u) = \sqrt{(\partial_u y_N(u))^2 + (1 + \hat{H} \partial_u y_N(u))^2}. \quad (11)$$

The interval computation simultaneously establishes a strictly positive lower bound for D , ensuring that all divisions appearing in the residual are well defined and rigorously controlled.

Finally, the residual is not certified merely at a finite collection of grid points. On each grid cell, a Taylor expansion with a rigorously bounded remainder propagates the interval estimates from the grid to the entire cell. Combining these bounds over one period, and using periodicity, yields a uniform estimate for every $u \in \mathbb{R}$.

Thus, the output of the interval-arithmetic stage is a pair of explicit inequalities

$$\|\mathcal{B}_{\mu,\beta}(y_N)\|_{L^\infty(\mathbb{R})} \leq \varepsilon, \quad D(u) \geq d_{\min} > 0 \quad \text{for all } u \in \mathbb{R}, \quad (12)$$

whose numerical justification is supplied by the validated computation. The bound includes the errors in evaluating the residual itself, rather than treating the numerical Hilbert transform or the sampled residual values as exact.

2.3.2 Formal verification in Lean

The second stage concerns the logical verification of the certificate. The purpose of Lean is not to repeat the interval-arithmetic computation or the floating-point training procedure. Instead, the numerical endpoints produced by Arb are represented as exact rational data, and the corresponding numerical certification statements are admitted as explicit assumptions. Lean verifies the mathematical deductions that connect these assumptions to the stated residual theorem.

In particular, the Babenko residual is defined formally, and the identities connecting the finite cosine representation, differentiation, the periodic Hilbert transform, and the numerical storage conventions are proved within Lean. Periodicity is then used to promote the certificate on a single period to a statement on all of $\mathbb{R}$. Under the stated numerical certification assumptions, the resulting theorem has the form

$$\forall u \in \mathbb{R}, \quad |\mathcal{B}_{\mu,\beta}(y_N)(u)| \leq \varepsilon, \quad (13)$$

with the exact rational tolerance given in (10).

The numerical inequalities supplied by the interval computation constitute the numerical trust boundary of the formal proof. As in standard computer-assisted proofs based on external interval-arithmetic computations, the validity of these numerical bounds rests on the validated-numerics software and its correct application. Lean does not independently recompute the bounds; rather, it checks that, once the numerical certification statements are admitted as assumptions, the claimed theorem follows through formally verified mathematical reasoning. Representing the endpoints as exact rationals prevents additional rounding ambiguity in the formal certificate, but does not itself verify the external numerical computation. This separation makes explicit the respective roles of the two components: interval arithmetic supplies the rigorous numerical estimates, while Lean checks the mathematical structure and logical deductions built upon them.

We emphasize that neither stage establishes the existence of an exact solution near y_N . The result certified here is the uniform residual estimate (9) for the explicit

polynomial, not a solution-error bound for the polynomial or the neural network. Upgrading this estimate to the existence of a nearby exact solution would additionally require quantitative invertibility estimates for the appropriately constrained linearized Babenko operator and control of the nonlinear remainder, as part of a Newton–Kantorovich or related contraction argument. That step lies beyond the scope of the present work.

3 Discussion

The results presented here demonstrate a route from machine-learning-based exploration of a nonlinear PDE to a rigorously certified and machine-checkable computational statement. The role of the neural network is primarily one of discovery: it provides a flexible representation with which to search nonlinear solution spaces and identify high-accuracy candidate solutions. The subsequent certification is deliberately separated from the training procedure. Once a candidate is selected, it is frozen as an explicit finite representation, and the mathematical claim concerns this representation rather than the neural network, optimizer or training history. This separation allows the exploratory flexibility of machine learning to be combined with the reproducibility and explicit error control of validated numerical computation.

The water-wave problem illustrates both the potential and the limitations of this approach. The same computational framework accesses qualitatively different high-amplitude regimes, including pure-gravity, pure-capillary and resonant gravity–capillary waves, but the search is not free of problem-specific mathematical information. Different solution families require appropriate physical, normalization and symmetry constraints, and continuation of high-amplitude Wilton ripples requires additional control of the relative amplitudes of the resonant Fourier modes. The neural network therefore does not replace mathematical structure; rather, it provides a flexible computational representation within which such structure can be imposed while leaving the detailed nonlinear profile to be determined by the governing equation. Developing methods that can infer useful continuation coordinates and constraints directly from the learned solution landscape would reduce this dependence on prior analytical knowledge.

A second distinction concerns numerical accuracy and mathematical existence. The certificate established here is a rigorous uniform bound on the residual of an explicit approximate solution. It shows that the reported near-machine-precision residual is not an artifact of floating-point roundoff, Fourier truncation, aliasing or finite sampling. It does not, by itself, prove the existence of an exact water-wave solution nearby. Such a result would require quantitative control of the linearized Babenko operator and of the nonlinear remainder, for example through a Newton–Kantorovich or related contraction argument. The certified residual obtained here provides one of the quantitative inputs required for such an existence proof and establishes a natural interface between machine-learning-based discovery and computer-assisted analysis.

Formal verification addresses a different source of uncertainty. At the accuracy considered here, the interval-arithmetic certification is itself a substantial computation involving hundreds of lines of validated numerical code and a sequence of analytic

estimates that would be difficult to audit completely by conventional inspection. Lean makes the mathematical identities and deductions connecting the numerical certificate to the final theorem machine-checkable. The present implementation nevertheless retains an explicit numerical trust boundary: Lean does not independently reproduce the Arb computation, and the validated numerical bounds supplied by the interval-arithmetic calculation remain trusted inputs to the formal layer. A more tightly integrated system in which validated numerical computations produce proof objects that can be checked directly by the proof assistant would further reduce this boundary.

An important feature of the workflow is that substantial parts of both the rigorous numerical implementation and the formalization were developed with assistance from a large language model. This illustrates a potentially broader role for generative AI in scientific computing: not only in producing numerical candidates, but also in helping construct the machinery required to validate them. The reliability of the final mathematical statement need not inherit the reliability of the generative model, provided that model-generated computations and arguments terminate in independently checkable certificates. In the present framework, interval arithmetic provides rigorous numerical enclosures and Lean checks the subsequent logical deductions.

More broadly, the approach is not specific to water waves or to PINNs. Its essential ingredients are a computational method capable of discovering accurate candidate solutions, an explicit representation to which validated numerics can be applied, rigorous control of discretization and continuum errors, and a formal layer capable of checking the deductions made from the resulting numerical bounds. These ingredients suggest a general architecture for trustworthy machine-learning-assisted scientific computing, particularly for nonlinear problems in which analytical descriptions are incomplete and numerical exploration plays an important role. Extending this architecture from residual certification to rigorous existence, stability and bifurcation results would provide a stronger connection between machine-assisted discovery and mathematical analysis.

4 Methods

4.1 PINN architecture

4.1.1 Neural representation

The unknown free surface is represented in conformal coordinates by a neural function of the periodic variable $u \in [-\pi, \pi]$. For the symmetric runs, periodicity and evenness are imposed by feeding the network the transformed coordinate $\cos u$. The implementation used for the plotted runs consists of three scalar fully connected multilayer perceptrons with tanh activations, initialized with Xavier-scaled truncated normal weights. Each scalar network has three hidden layers of width 30. The networks are combined as

$$y_N(u) = s N_0(\cos u), \quad T_{1,N}(u) = s N_1(\cos u) \sin u, \quad T_{2,N}(u) = s N_2(\cos u), \quad (14)$$

where s is the amplitude scale used in a given continuation step. This construction makes y_N and $T_{2,N}$ even and $T_{1,N}$ odd, matching the crest-symmetric geometry. The auxiliary variables $T_{1,N}$ and $T_{2,N}$ represent the two components of the normalized tangent/curvature expression,

$$T_1 = \frac{y_u}{\sqrt{y_u^2 + (1 + \hat{H}y_u)^2}}, \quad T_2 = \frac{1 + \hat{H}y_u}{\sqrt{y_u^2 + (1 + \hat{H}y_u)^2}}, \quad (15)$$

so that the surface-tension term can be evaluated stably through first derivatives of the auxiliary outputs.

The wave speed is optimized together with the network weights through a bounded scalar parameterization

$$c_N = c_0 + \eta_c \tanh \rho_c, \quad (16)$$

where c_0 is the continuation seed and η_c is the allowed local search radius. In the Wilton continuation runs in which surface tension is trained, the same bounded parametrization is used for β_N . Fixed-regime benchmark runs set the corresponding physical parameter to its prescribed value.

4.1.2 Residual, Hilbert transform and loss

All derivatives of the neural output are obtained by automatic differentiation in JAX using double precision. The periodic Hilbert transform is evaluated numerically on a dense nonuniform grid of 15,000 sample points. The singular integral is split into a local contribution around the evaluation point, a close-field correction and an outer contribution evaluated by Gaussian quadrature. The same Hilbert-transform routine is used for y_N , $y_{N,u}$, derivatives of y_N^2 and derivatives of the auxiliary variables entering the surface-tension term.

Training minimizes a weighted loss

$$\mathcal{L} = w_{\text{eq}} \sum_{u_i \in \mathcal{C}} \omega_i \|\mathcal{R}_N(u_i)\|^2 + w_{\text{der}} \sum_{v_j \in \mathcal{D}} \nu_j |\partial_u \mathcal{B}(y_N)(v_j)|^2 + \mathcal{L}_{\text{aux}}. \quad (17)$$

Here $\mathcal{R}_N = (e_1, e_2, e_3)$ consists of the Babenko residual, $e_1 = \mathcal{B}(y_N)$, and the two algebraic residuals defining $T_{1,N}$ and $T_{2,N}$. The auxiliary loss $\mathcal{L}_{\text{aux}}$ fixes the amplitude, removes vertical translation through a mean-zero condition and, when needed, enforces or breaks prescribed symmetry conditions. The active runs use $w_{\text{eq}} = 0.1$ with residual-component weight one; the normalization and mean constraints are kept on throughout training.

4.1.3 Collocation and optimization

The collocation sets are resampled during optimization. In the runs used here, each optimization step combines fixed and moving collocation points: 251 fixed equation points, 201 fixed derivative points, two groups of 71 points concentrated near the endpoints and 121 moving points for each of the equation and derivative residuals. The sampling density is biased toward regions where high-amplitude waves develop

sharp features, especially near crests and troughs. After a training pass, the point distribution can be updated from the residual itself so that later passes concentrate effort where the current approximation is least accurate.

The optimizer is KFAC in JAX, with adaptive learning rate, momentum and damping, minimum damping 10^{-14} and a typical training horizon of 15,000 iterations per continuation step. New branches are followed by warm starting from a previously trained profile and allowing the speed, and when appropriate the surface tension, to move within a prescribed local window. The trained state is saved as both a weight checkpoint and a MATLAB file containing x , y_N , $\hat{H}y_N$, derivatives, residual values and the loss history.

4.2 Second-stage training

After the first training stage produced a candidate wave, the second stage targeted its remaining structured residual through successive additive corrections. Let $Y_p = (y_p, T_{1,p}, T_{2,p})$ denote the previously trained fields, frozen at the start of a refinement pass. The corrected approximation is

$$Y_N(u) = Y_p(u) + \varepsilon_p \Phi_\theta(u), \quad (18)$$

where $Y_N = (y_N, T_{1,N}, T_{2,N})$, Φ_θ is a three-component correction network, and ε_p sets its amplitude scale. Write $\mathcal{R}(Y)$ for the three-field Babenko residual operator and $\mathcal{R}_N = \mathcal{R}(Y_N) = (e_1, e_2, e_3)$. Here e_1 is the main equation residual, while e_2 and e_3 enforce the definitions of $T_{2,N}$ and $T_{1,N}$, respectively. When those definitions hold, $e_1 = \mathcal{B}(y_N)$, where $\mathcal{B}$ is the scalar Babenko operator in Eq. (4).

The correction ansatz is motivated by linearization about the frozen fields [4, 5]:

$$\mathcal{R}(Y_p + \varepsilon_p \Phi_\theta) \simeq \mathcal{R}(Y_p) + \varepsilon_p D\mathcal{R}(Y_p) \Phi_\theta. \quad (19)$$

To cancel the existing residual to first order, the correction should therefore approximately satisfy

$$\varepsilon_p D\mathcal{R}(Y_p) \Phi_\theta \simeq -\mathcal{R}(Y_p). \quad (20)$$

The linearized equation explains the correction strategy; optimization evaluates the full nonlinear residual $\mathcal{R}_N$.

The correction networks used sinusoidal activations and a tunable first-layer frequency scale. We chose the frequency scale of the correction to y_N from the prominent frequencies in the remaining residual, and chose ε_p according to its magnitude. These choices guide the scales represented by the network; they do not constrain its output to a single Fourier mode or make ε_p equal to a residual norm. The input $\cos u$ makes the outputs periodic and even, while a factor of $\sin u$ imposes odd parity on the $T_{1,N}$ correction. After one correction was trained, its contribution was incorporated into the frozen base and a further, smaller-scale correction was trained. The capillary parameter remained fixed, while the wave speed was allowed a small adjustment.

The refinement stage evaluated the periodic Hilbert transform by FFT on full uniform grids rather than by the nonuniform quadrature used during discovery. Neural-field derivatives were computed by automatic differentiation. Shifting the uniform grid

by fractions of a grid spacing sampled the residual between earlier collocation points while preserving the periodic grid required by the FFT. Later refinement passes used finer grids.

4.2.1 Gradient-normalized residual weighting

The second-stage refinement also employed a gradient-normalized residual loss. A standard mean-square residual,

$$\sum_j |\mathcal{R}(Y_N)(u_j)|^2,$$

weights the residual in absolute terms. When the solution contains steep or rapidly varying regions, however, the natural scale of the equation can vary strongly across the domain, causing such regions to dominate the optimization. We therefore normalize the residual by a local scale determined from spatial derivatives of the frozen base fields, leading schematically to

$$\sum_j \left| \frac{\mathcal{R}(Y_N)(u_j)}{\text{local derivative scale at } u_j} \right|^2.$$

In this sense, the loss measures a locally scaled residual rather than a purely absolute one.

For the Babenko system, the second-stage loss retained the component-wise weighted residual structure used during training. Let $\mathcal{C}$ denote the collocation points for $\mathcal{R}_N$ and $\mathcal{D}$ those used for the derivative of the main equation. To balance regions with different levels of spatial variation, the residuals were divided by scales S_{eq} and S_{der} constructed from spatial derivatives of the frozen fields Y_p . These scales combine derivatives of y_p , their Hilbert transforms, and capillary contributions involving $T_{1,p}$ and $T_{2,p}$. Each scale is normalized by its grid average and clipped to prevent extreme reweighting, while S_{der} places greater emphasis on the curvature of the base fields. The scales are recomputed whenever the grid is changed and are held fixed during parameter differentiation, preventing the correction network from reducing the loss merely by altering its normalization.

With component-specific collocation weights $\omega_{i\ell}$ and derivative weights ν_j , the refinement loss is

$$\mathcal{L} = w_{\text{eq}} \sum_{u_i \in \mathcal{C}} \sum_{\ell=1}^3 \omega_{i\ell} \left| \frac{e_\ell(u_i)}{S_{\text{eq}}(u_i)} \right|^2 + w_{\text{der}} \sum_{v_j \in \mathcal{D}} \nu_j \left| \frac{\partial_u e_1(v_j)}{S_{\text{der}}(v_j)} \right|^2 + \mathcal{L}_{\text{aux}}. \quad (21)$$

The derivative term was introduced during refinement and strengthened for the final correction in order to suppress rapidly varying residual errors. The weights $\omega_{i\ell}$ emphasize the periodic endpoints and, in intermediate passes, selected residual peaks; they are normalized so that the total weight assigned to each residual component remains fixed. The derivative weights ν_j are uniform. The term $\mathcal{L}_{\text{aux}}$ contains the retained

amplitude and crest–trough separation penalties. The derivative term acts on e_1 , the main equation evaluated with the current auxiliary fields, and agrees with $\partial_u \mathcal{B}(y_N)$ when their defining residuals vanish.

After the final refinement pass, the corrected fields were evaluated on shifted grids before Fourier projection and Newton polishing. Rigorous certification was then carried out separately on the resulting frozen trigonometric polynomial.

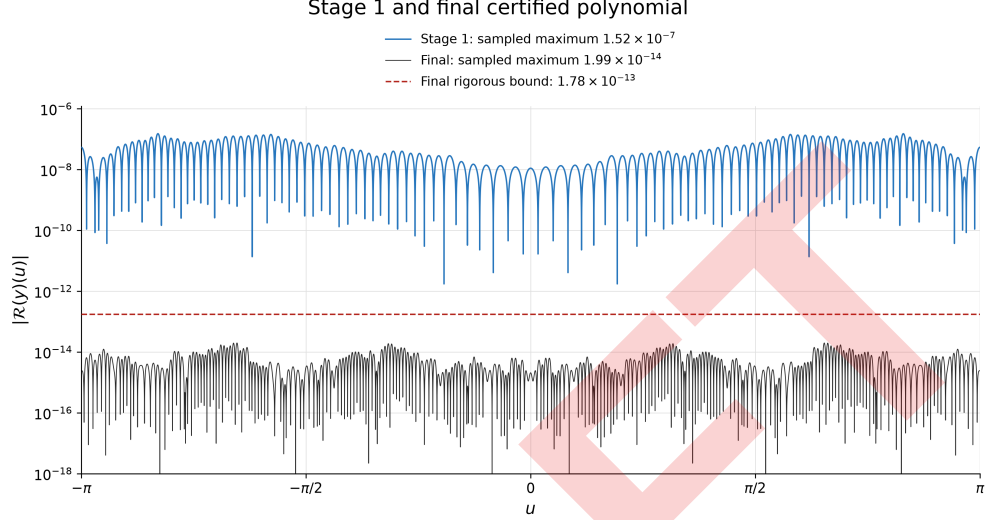


Fig. 6 Reduction of the scalar Babenko residual. Absolute residual as a function of the conformal coordinate u for the first-stage neural profile (blue) and the final trigonometric polynomial (black), obtained after successive neural corrections, Fourier projection, and Newton polishing. Both curves evaluate the scalar Babenko equation with the auxiliary fields eliminated, at the physical parameters of the corresponding profile. The curves show pointwise numerical evaluations; the dashed red line gives the rigorous uniform residual bound for the final polynomial.

Figure 6 compares the scalar Babenko residual of the first-stage profile with that of the final trigonometric polynomial, illustrating the reduction achieved through successive neural corrections, Fourier projection, and Newton polishing. The rigorous uniform bound for the final polynomial is shown separately.

4.3 Computer-assisted accuracy proof

We certify a uniform bound on the Babenko residual of a fixed finite profile, as follows:

Let

$$y_N(u) = \frac{a_0}{2} + \sum_{k=1}^N a_k \cos(ku), \quad (22)$$

be a fixed finite even cosine series and let $\mu, \beta > 0$ be given rational parameters. Define the residual of the Babenko equation with surface tension by

$$\begin{aligned} \mathcal{R}(y) = -\mathcal{B}(y) = & \mu \hat{H}[y_u] - y - y \hat{H}[y_u] - \hat{H}[yy_u] \\ & + \beta \partial_u \left(\frac{y_u}{D} \right) - \beta \hat{H} \left[\partial_u \left(\frac{1 + \hat{H}[y_u]}{D} \right) \right]. \end{aligned} \quad (23)$$

where

$$D(u) = \sqrt{y_u^2 + \left(1 + \hat{H}[y_u]\right)^2}, \quad (24)$$

and $\hat{H}$ denotes the 2π -periodic Hilbert transform, with convention

$$\hat{H}[\cos(ku)] = \sin(ku), \quad \hat{H}[\sin(ku)] = -\cos(ku), \quad k \geq 1. \quad (25)$$

The goal is to establish $D(u) \geq d_* > 0$ for every $u \in \mathbb{R}$ and $\|\mathcal{R}(y_N)\|_{L^\infty(\mathbb{R})} \leq \epsilon$ for explicit certified constants d_* and ϵ .

The rigorous proof is split between Julia and Lean 4. A Julia verifier performs the substantive computation using rigorous interval ball arithmetic: it treats the final computed wave as a fixed trigonometric polynomial and establishes a uniform bound on its Babenko residual together with a positive lower bound on the capillary denominator, all with certified error enclosures. A Lean development provides the logical scaffolding: it formalizes the relevant operators and reusable estimates, imports the two final numerical conclusions as explicit assumptions, and checks the deduction of the paper’s residual theorem. The Lean kernel checks this conditional deduction without recomputing the numerical enclosures. Once the polynomial has been frozen, the network weights and training trajectory play no role in the proof. Source code, archived run evidence, build instructions, and the supplementary mathematical account are indexed in the [paper repository](#).

4.3.1 Computer-assisted interval certification of the numerical residual

After neural training and continuation, we project the profile onto an even cosine series and polish its coefficients with a finite-dimensional Newton iteration. The resulting polynomial has the convention in Eq. (22). Its stored binary coefficients and parameters are interpreted as exact dyadic rationals and fixed before validation. The certified statement concerns this explicit polynomial and these exact parameter values. From this point onward, the certification is independent of the neural network. The normalization and computational settings for the reported profile are collected in Section 4.3.3.

Julia and Arb compute outward-rounded interval enclosures for the nonlinear residual, add analytic corrections for unresolved Fourier modes, and use Taylor’s theorem to pass from a grid to the whole torus.

The Julia package `BabenkoRigorous.jl` encloses every numerical operation with outward-rounded Arb balls. It first constructs finite Fourier representations of y_N ,

$y_{N,u}$, $\hat{H}[y_{N,u}]$ and their polynomial products. Differentiation and the periodic Hilbert transform act mode by mode, while multiplication uses finite coefficient convolution, with no projection of newly generated modes back to degree N .

Nonlinear capillary quantities require a second representation. With D as defined in Eq. (24), the verifier forms

$$G(u) = \frac{1 + \hat{H}[y_{N,u}](u)}{D(u)}. \quad (26)$$

At each node of a periodic grid, a Taylor record contains interval enclosures of a quantity and its derivatives, together with global derivative suprema. Product, reciprocal and square-root rules propagate these records. A grid lower scan combined with off-grid derivative control first proves $D(u) \geq d_* > 0$ over the entire period; only then are D^{-1} and the capillary fractions evaluated.

The Hilbert-transform work is divided into two pathways.

Path A: finite trigonometric algebra. The input has a finite Fourier representation, so $\hat{H}$ acts mode by mode using Eq. (25). No sampling or aliasing correction is needed for this operation.

Path B: certified sampled transform. The input is represented by interval-enclosed values and derivatives on a uniform grid, rather than by a finite coefficient vector. A discrete Hilbert transform is evaluated by certified convolution, and a rigorous Fourier-tail correction is added to enclose the continuous transform.

Path B is essential for $\hat{H}[\partial_u G]$, because the square root and reciprocal in Eqs. (24) and (26) generally give G an infinite Fourier series. The current Julia implementation also routes $\hat{H}[y_N y_{N,u}]$ through Path B for uniformity, although $y_N y_{N,u}$ is itself a finite trigonometric polynomial of degree at most $2N$.

If $\|F^{(p)}\|_\infty \leq C_p$ for $p \geq 2$, the correction at each grid node is bounded by

$$\varepsilon_{\text{alias}}(F) = \frac{4C_p}{(p-1)(N_{\text{grid}}/2-1)^{p-1}}. \quad (27)$$

Arb encloses arithmetic roundoff; the aliasing correction is a separate analytic estimate supplied by the verifier.

The full residual is assembled from these interval records using the same terms as Eq. (23). Global derivative bounds are obtained from the finite Fourier coefficients for trigonometric polynomials and propagated through products, reciprocals and square roots using differentiation rules and interval arithmetic. The positive lower bound on D controls the denominators arising in these estimates. For sampled Hilbert-transform terms, Fourier-decay estimates provide the additional derivative bounds required by the Taylor remainder.

For grid nodes u_j separated by $h = 2\pi/N_{\text{grid}}$, let $I_{j,m}$ enclose the m -th derivative of the residual at u_j , and set $U_{j,m} = \sup_{x \in I_{j,m}} |x|$. A global bound $C_{M+1} \geq$

$\|\partial_u^{M+1}\mathcal{B}(y_N)\|_\infty$ gives, for $u_j \leq u \leq u_j + h$,

$$|\mathcal{B}(y_N)(u)| \leq \sum_{m=0}^M \frac{U_{j,m} h^m}{m!} + \frac{C_{M+1} h^{M+1}}{(M+1)!}. \quad (28)$$

The Taylor order M is chosen together with the grid size N_{grid} so that the remainder bound is sufficiently small to achieve the desired certification accuracy. Taking the outward-rounded maximum over all cells then proves a uniform bound over the entire period. The denominator scan uses the corresponding lower-bound argument.

The production computation uses a sharded implementation: local Taylor data are streamed to disk, Hilbert-transform derivative columns are processed separately, and the final residual is scanned shard by shard.

The implementation, input coefficients, run logs and reproduction commands are in the [Julia directory](#). The detailed interval estimates, including the discrete-transform correction and the off-grid argument, are in the [Supplementary Methods](#).

4.3.2 Formal definitions, analytic estimates and numerical certificate in Lean

The Lean project **WaveCAP** defines the mathematical operators explicitly. The periodic Hilbert transform is defined as a Fourier multiplier, with multiplier $-i \operatorname{sgn}(n)$ for nonzero modes and zero for the constant mode. The development also defines the finite profile, its derivative, the capillary quantities D and G , and the residual in Eq. (23).

The reusable library formalizes the mathematical estimates underlying the Julia calculation. These include the finite Hilbert-transform algebra of Path A, the sampled-transform aliasing estimate of Path B, derivative bounds for products and compositions, and the Taylor grid-to-continuum estimate. It also formalizes the mathematical contracts for Taylor records and their arithmetic operations, the denominator and capillary estimates, and the assembly and periodicity of the residual. These general results are independent of the particular numerical profile.

For the reported profile, the generated certificate stores the coefficients, parameters, grid metadata and final interval endpoints as exact rational constants. A half-DC adapter reconciles Julia's $a_0/2$ convention in Eq. (22) with Lean's direct zeroth coefficient.

The two numerical conclusions supplied by Julia enter Lean as explicit assumptions. The assumption `cert_arb_dmin_bound` gives the positive global lower bound on D . The assumption `cert_arb_sup_bound_le_eps0` gives the residual bound on one period, conditional on the denominator bound. Lean then checks the deductions from these assumptions: periodic extension to every real argument, exact comparisons with the reported decimal constants, and agreement of the absolute residual bounds under the implementation and manuscript sign conventions.

The present certificate-specific proof does not replay the Arb computation or instantiate every generic theorem with the intermediate numerical records from that run. Table 1 distinguishes the general formal results from the deductions using the imported numerical bounds.

Table 1 Lean proof components and numerical inputs. The first six rows summarize general mathematical results and data-structure contracts. The final three rows describe how the numerical bounds supplied by Julia are used to derive the reported estimates.

Component	Formal result or role
Finite Hilbert transform	Proves the mode-by-mode action on trigonometric polynomials and the identity $H[y_u] = H_y$.
Sampled Hilbert transform	Proves the DFT aliasing estimate and cot-kernel equivalence, and states the correctness contract for the interval transform and its Taylor lift.
Derivative calculus	Proves product and composition estimates used to bound higher derivatives.
Taylor records and continuum bounds	Formalizes Taylor-record semantics and arithmetic, including reciprocal and square-root recurrences, and proves the Taylor remainder and grid-to-continuum estimates.
Denominator and capillary quantities	Formalizes the positivity calculation and proves derivative estimates for D and G .
Residual assembly	Combines Path-A and Path-B records, proves periodicity, and extends a one-period bound to every real argument.
Certificate inputs	Accepts the global denominator bound and the one-period residual bound as two explicit numerical assumptions supplied by Julia.
Displayed bounds	Checks exact rational comparisons between the certificate endpoints and the reported decimal constants.
Manuscript residual	Proves that the implementation and manuscript residuals have opposite signs and therefore identical absolute values.

The audit module lists the assumptions used by the final theorems. Besides Lean’s standard logical axioms, the residual theorem depends on the two numerical assumptions supplied by Julia, while the denominator theorem depends only on the denominator assumption.

The resulting statement is a certified uniform residual bound for the fixed polynomial at the stated exact parameter values. An existence theorem for a nearby exact water wave would require a further validated inverse estimate. The [Lean directory](#) contains the formal definitions, theorem modules, certificate deductions, axiom audit and pinned build instructions.

4.3.3 Certified profile, computational settings and bounds

For the candidate reported in the main text, the certified input is the polynomial in Eq. (22), with stored binary coefficients interpreted as exact dyadic rationals. In the normalization $g = 1$, the certified parameters are the stored binary64 values $\mu_0 = \text{fl}_{64}(\mu_*)$ and $\beta_0 = \text{fl}_{64}(\beta_*)$, likewise interpreted exactly. The coefficient and parameter values are supplied with the archived input in the paper repository. The polynomial degree is

$$N = 150. \tag{29}$$

The rigorous calculation uses

$$N_{\text{grid}} = 2,000,000, \quad M = 6, \tag{30}$$

at 128-bit Arb precision.

For these fixed inputs, the computation yields

$$\|\mathcal{B}_{\mu_0, \beta_0}(y_{150})\|_{L^\infty(\mathbb{R})} \leq 1.78 \times 10^{-13} \quad (31)$$

and, independently,

$$D(u) \geq 0.3446588 > 0 \quad \text{for every } u \in \mathbb{R}. \quad (32)$$

The bound (31) includes the propagated Arb rounding error, the discrete-Hilbert aliasing error, the nonlinear derivative bounds, and the Taylor remainder controlling the intervals between grid points. It is therefore a rigorous uniform residual certificate for the explicit polynomial y_{150} at the stated exact parameter values.

4.4 Reproducibility and file map

The inputs, logs and certificate artifacts for the certified run are archived in

`BabenkoRigorous/results/phase4_scalar_n150_20260827_2m_20260829`

within the paper repository. These include the fixed input coefficients and parameters, reproduction scripts, Arb endpoint dumps, and the generated Lean certificate, together with checksums. The installed half-DC certificate was verified to match the generated artifact byte for byte. The certificate-specific theorems and assumption audit were checked with the pinned Lean and Mathlib versions recorded in the repository.

The certified interval calculation was performed on an NYU CIMS compute server. Table 2 summarizes the execution times and memory use recorded in the archived logs. The eight workers ran concurrently, so their wall times are not summed. The slowest worker followed by the three serial stages gives an estimated production wall time of approximately 2 hours 50 minutes after warmup. The sum of the individual worker memory peaks was 61.5 GB; this does not represent a measured simultaneous peak for the full run.

Table 3 reproduces the numerical summary printed by the final `s3` stage. The displayed digits summarize the archived Arb enclosures; the Lean certificate uses exact rational endpoints. The top-level run log records `ALL_STAGES_COMPLETE`.

Table 2 Execution time and memory use. Wall times are taken from the system timing logs. Peak RSS denotes peak resident memory per process.

Stage	Threads	Wall time	Peak RSS (GB)
Worker 1	4	1 h 57 min 25 s	5.019
Worker 2	4	2 h 14 min 58 s	9.242
Worker 3	4	1 h 56 min 25 s	4.972
Worker 4	4	2 h 07 min 07 s	9.287
Worker 5	4	2 h 13 min 04 s	9.324
Worker 6	4	1 h 58 min 34 s	5.088
Worker 7	4	2 h 09 min 00 s	9.267
Worker 8	4	2 h 13 min 52 s	9.283
Merge	8	1 min 22.08 s	0.481
Hilbert-transform stage	8	21 min 33.92 s	5.149
Final residual scan (s3)	8	11 min 49.78 s	0.477

Table 3 Final-stage numerical summary. Values transcribed from the archived s3 log.

Log entry	Reported value
Bound midpoint	$+1.9928311263803702 \times 10^{-14}$
Bound radius	$1.5799937398052128 \times 10^{-13}$
Bound interval: lower endpoint	$-1.3807106271671758 \times 10^{-13}$
Bound interval: upper endpoint	$+1.7792768524432498 \times 10^{-13}$
Residual upper bound	$1.7792768524432498 \times 10^{-13}$
Denominator lower bound	$3.4465883518530394 \times 10^{-1}$

5 Data and code availability

The `.mat` files used to generate the figures are currently stored in the manuscript run folders, including the pure-gravity PINN runs and the gravity-capillary neural-network runs. A curated archive of selected trained profiles, checkpoints, residual diagnostics and the figure-generation summary table will be deposited before submission and linked from the paper’s GitHub repository, <https://github.com/michalshavitNYU/Rigorously-certified-water-waves>.

The repository currently contains the Julia interval-arithmetic code, numerical certificates and run records for the computer-assisted residual bound, together with the Lean formalization, pinned software environments and instructions for checking the certificate.

Declarations

Competing interests The authors declare no competing interests.

References

- [1] Nalimov, V.I.: The cauchy-poisson problem. *Dinamika Sploshn. Sredy* **18**, 104–210 (1974)
- [2] Lannes, D.: *The Water Waves Problem: Mathematical Analysis and Asymptotics* vol. 188. American Mathematical Society, Providence, RI (2013)
- [3] Raissi, M., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics* **378**, 686–707 (2019) <https://doi.org/10.1016/j.jcp.2018.10.045>
- [4] Wang, Y., Lai, C.-Y.: Multi-stage neural networks: Function approximator of machine precision. *Journal of Computational Physics* **504**, 112865 (2024)
- [5] Wang, Y., Léger, T., Lai, C.-Y., Buckmaster, T.: Resolving Sharp Gradients of Unstable Singularities to Machine Precision via Neural Networks (2025). <https://doi.org/10.48550/arXiv.2511.22819> . <https://arxiv.org/abs/2511.22819>
- [6] Toland, J.F.: Stokes waves. *Topological Methods in Nonlinear Analysis* **7**, 1–48 (1996)
- [7] Crapper, G.D.: An exact solution for progressive capillary waves of arbitrary amplitude. *Journal of Fluid Mechanics* **2**(6), 532–540 (1957) <https://doi.org/10.1017/S0022112057000348>
- [8] Wilton, J.R.: On ripples. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* **29**(173), 688–700 (1915) <https://doi.org/10.1080/14786440508635350>
- [9] Ovsyannikov, L.V.: *Dynamika sploshnoi sredy*, lavrentiev institute of hydrodynamics. *Sib. Branch Acad. Sci. USSR* **15**, 104 (1973)
- [10] Tanveer, S.: Singularities in water waves and rayleigh-taylor instability. *Proceedings of the Royal Society of London. Series A* **435**(1893), 137–158 (1991)
- [11] Dyachenko, A.I., Zakharov, V.E., Kuznetsov, E.A.: Nonlinear dynamics of the free surface of an ideal fluid. *Plasma Physics Reports* **22**(10), 829–840 (1996)
- [12] Stokes, G.G.: On the theory of oscillatory waves. *Transactions of the Cambridge Philosophical Society* **8**, 441–455 (1847)
- [13] Hillebrecht, B., Unger, B.: Rigorous a posteriori error bounds for PDE-defined PINNs. *IEEE Transactions on Neural Networks and Learning Systems* **36**(1), 1583–1593 (2025) <https://doi.org/10.1109/TNNLS.2023.3335837> [arXiv:2210.03426](https://arxiv.org/abs/2210.03426)

- [14] Eiras, F., Bibi, A., Bunel, R.R., Dvijotham, K.D., Torr, P., Kumar, M.P.: Efficient error certification for physics-informed neural networks. In: Proceedings of the 41st International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 235, pp. 12318–12347. PMLR, ??? (2024). <https://proceedings.mlr.press/v235/eiras24a.html>
- [15] Tanaka, K., Yatabe, K.: Learn and Verify: A Framework for Rigorous Verification of Physics-Informed Neural Networks (2026). <https://doi.org/10.48550/arXiv.2601.19818> . <https://arxiv.org/abs/2601.19818>
- [16] George, R.J., Cruden, J., Adkisson, W., Zhong, X., Zhang, H., Anandkumar, A.: TorchLean: Formalizing Neural Networks in Lean. Version 2 (2026). <https://doi.org/10.48550/arXiv.2602.22631> . <https://arxiv.org/abs/2602.22631v2>

DRAFT